SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Manoj M	**Reg. No.:**	192511060
Section / Batch:	CSE	**Date:**	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Maximum Sum Circular Subarray:

Given a circular array of integers, find the maximum possible sum of a non-empty subarray. The subarray may wrap around the end of the array. The objective is to consider both normal and circular cases. Use Dynamic Programming to compute the result efficiently. Input Format: n = number of elements arr[] = elements Sample Input: n = 5 arr = 8 -1 3 4 Expected Output: Max Sum = 15

Code of Conduct

I _____Manoj (192511060)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

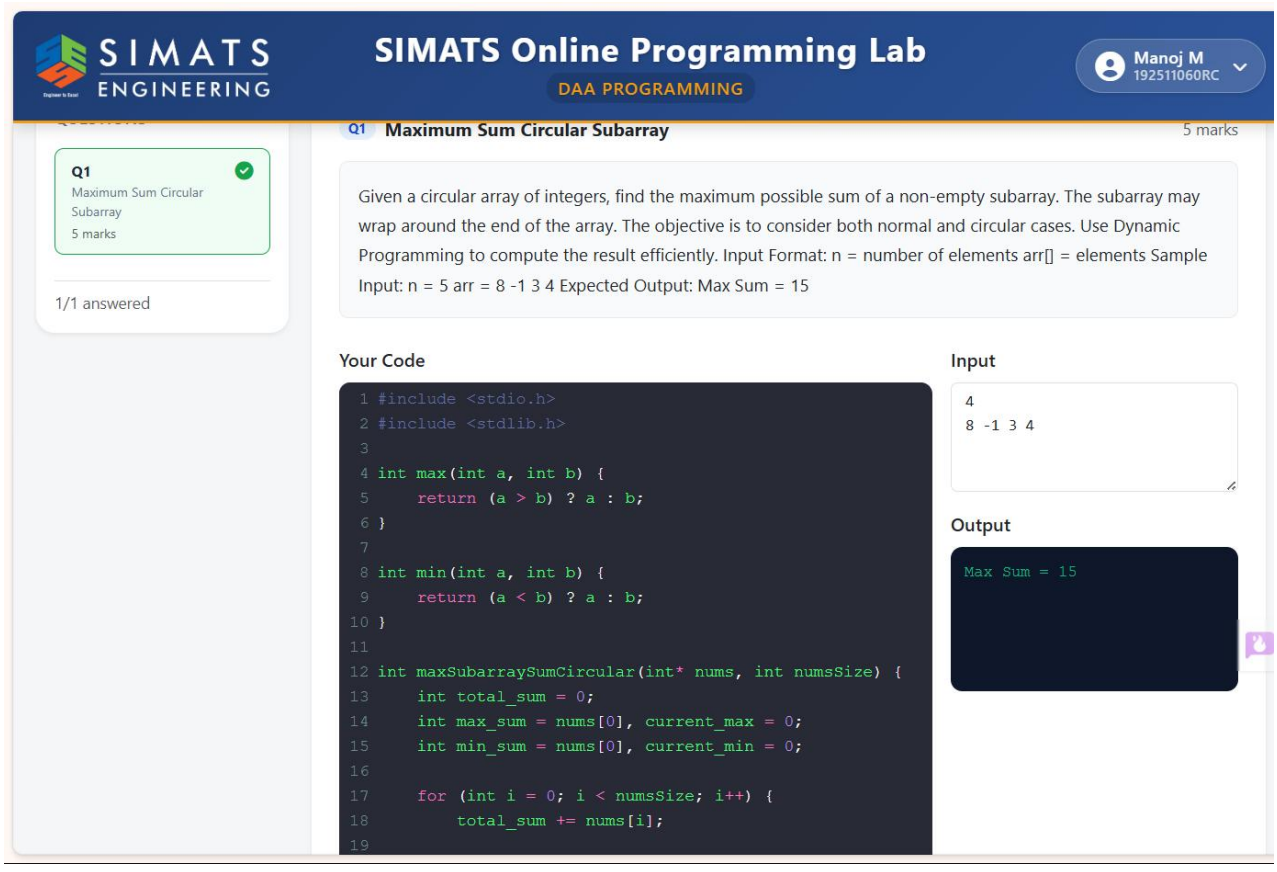
Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				

Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Code:



Algorithm:

Algorithm: Maximum Sum Circular Subarray

1. Initialize totalSum, maxSum, minSum, and current maximum/minimum sums.
2. Traverse the array and use **Kadane's Algorithm** to find:
    - maxSum → maximum normal subarray sum.
    - minSum → minimum subarray sum.
    - totalSum → sum of all elements.
3. Calculate the circular maximum sum as:
   circularSum = totalSum - minSum
4. If all elements are negative (maxSum < 0), return maxSum.
5. Otherwise, return the maximum of maxSum and circularSum.
6. **Time Complexity:** O(n)
   Space Complexity: O(1)

Example:

arr = [8, -1, 3, 4]

Normal maximum = 14 (8 + -1 + 3 + 4)

Circular maximum = 15 (3 + 4 + 8)

Output: Max Sum = 15